

Paytm SSE Round 1 - Low-Level Design & Concurrency

1. The Double Booking Problem

Problem: "How do you ensure two users don't book the same movie seat at the exact same millisecond?"

This is the bread-and-butter problem for any payments company like Paytm. Whether it's movie seats, flight tickets, or wallet balances — the core challenge is the same: **preventing two concurrent transactions from claiming the same resource.**

The Race Condition

```
Timeline (both users see Seat A7 as available): User 1 User 2 ■■■■■■■■ ■■■■■■■■ READ: Seat A7 = AVAILABLE
READ: Seat A7 = AVAILABLE UPDATE: Seat A7 = BOOKED (by User 1) UPDATE: Seat A7 = BOOKED (by User 2) ← DOUBLE
BOOKING!
```

Both users read the seat as available before either could write. Classic TOCTOU (Time of Check to Time of Use) bug.

Solution 1: Pessimistic Locking (DB-Level `FOR UPDATE`)

Idea: "Lock the resource before even reading it. No one else can touch it while I'm working."

```
BEGIN TRANSACTION; -- This LOCKS the row. Any other transaction trying to SELECT FOR UPDATE -- on the same
row will WAIT until this transaction commits or rolls back. SELECT * FROM seats WHERE seat_id = 'A7' AND
show_id = 123 FOR UPDATE; -- Check if still available (guaranteed no one else can change it) -- If available:
UPDATE seats SET status = 'BOOKED', user_id = 456 WHERE seat_id = 'A7' AND show_id = 123; COMMIT;
```

How it solves the problem:

```
User 1 User 2 ■■■■■■■■ ■■■■■■■■ SELECT ... FOR UPDATE → Locks row A7 SELECT ... FOR UPDATE → BLOCKED
(waiting) UPDATE → Seat booked for User 1 COMMIT → Lock released Lock acquired → Reads seat → ALREADY
BOOKED → Returns "Sorry, seat taken"
```

Pros:

- Simple and bulletproof
- Guaranteed consistency

Cons:

- Performance hit — locks block other transactions
- Risk of **deadlocks** if multiple rows are locked in different orders
- Doesn't scale well under high concurrency (thousands of users booking simultaneously)

When to use: Low-to-medium concurrency. Critical operations where correctness is more important than speed (e.g., bank transfers, seat booking).

Solution 2: Optimistic Locking (Version-Based)

Idea: "Don't lock anything. Let everyone read freely. But when writing, check if someone else changed it first."

Add a `version` column to the table:

```
CREATE TABLE seats ( seat_id VARCHAR(10), show_id INT, status VARCHAR(10), user_id INT, version INT DEFAULT 0
-- ← This is the magic column );
```

The booking flow:

```
-- Step 1: Read the seat (no lock) SELECT seat_id, status, version FROM seats WHERE seat_id = 'A7' AND
show_id = 123; -- Returns: status=AVAILABLE, version=0 -- Step 2: Update ONLY IF version hasn't changed
UPDATE seats SET status = 'BOOKED', user_id = 456, version = version + 1 WHERE seat_id = 'A7' AND show_id =
123 AND version = 0; -- ↑ This WHERE clause is the key! -- Step 3: Check rows affected -- If 1 row updated →
SUCCESS -- If 0 rows updated → Someone else already changed it → RETRY or FAIL
```

How it solves the problem:

```
User 1 User 2 ■■■■■■■■ ■■■■■■■■ READ: version=0, status=AVAILABLE READ: version=0, status=AVAILABLE
UPDATE WHERE version=0 → 1 row affected → version becomes 1, status=BOOKED UPDATE WHERE version=0 → 0 rows
affected! (version is now 1, not 0) → "Booking failed, please try again"
```

In JPA/Hibernate:

```
@Entity public class Seat { @Id private String seatId; private String status; @Version // ← Hibernate handles
versioning automatically private Integer version; }
```

Hibernate throws `OptimisticLockException` when the version doesn't match.

Pros:

- No locks → much better throughput
- No deadlocks possible
- Scales well under high read, low write contention

Cons:

- Requires retry logic on failure
- Under very high write contention, many retries can happen (thrashing)

When to use: High concurrency with low collision probability. E-commerce carts, content updates, inventory (where most users are looking at different items).

Solution 3: The Paytm-Scale Approach (Distributed Locking with Redis)

For truly high concurrency (Paytm handles millions of transactions), DB-level locking becomes a bottleneck. Use **Redis distributed locks**:

```
// Try to acquire a lock on this specific seat String lockKey = "lock:seat:" + showId + ":" + seatId; boolean
acquired = redis.set(lockKey, uniqueId, "NX", "EX", 5); // NX = Only set if not exists // EX = Expire in 5
seconds (safety net) if (acquired) { try { // Check and book the seat Seat seat =
seatRepo.findByShowAndSeat(showId, seatId); if (seat.isAvailable()) { seat.book(userId); seatRepo.save(seat);
} } finally { // Release lock (only if we still own it) redis.eval("if redis.call('get',KEYS[1]) == ARGV[1]
then " + "return redis.call('del',KEYS[1]) else return 0 end", lockKey, uniqueId); } }
```

Summary: When to Use Which?


```
public class SlidingWindowLog { private final int maxRequests; private final long windowSizeMs; private final Queue<Long> requestLog = new LinkedList<>(); public synchronized boolean allowRequest() { long now = System.currentTimeMillis(); long windowStart = now - windowSizeMs; // Remove expired entries while (!requestLog.isEmpty() && requestLog.peek() <= windowStart) { requestLog.poll(); } if (requestLog.size() < maxRequests) { requestLog.offer(now); return true; } return false; } }
```

Pros: Exact, no boundary burst problem.

Cons: Memory-heavy (stores every timestamp). Not practical for high-traffic APIs.

Algorithm 3: Token Bucket (Industry Standard)

This is what most real-world systems use (AWS API Gateway, Stripe, etc.).

Concept:

- Imagine a bucket that holds tokens (max capacity = `maxTokens`)
- Tokens are added at a constant rate (e.g., 10 tokens/second)
- Each request consumes one token
- If the bucket is empty → request rejected
- The bucket can accumulate tokens up to `maxTokens`, allowing short bursts

```
■■■■■■■■■■■■■■■■■■■■ ■ Token ■ ← Tokens added at fixed rate (refill) ■ Bucket ■ ■ ●●●●●●●● ■ ← Max capacity: 10 tokens ■ (8 tokens) ■ ■■■■■■■■■■■■■■■■■■■■ ■ Request comes in → take 1 token → 7 tokens left → ALLOWED Next request → take 1 token → 6 tokens left → ALLOWED ... Bucket empty → REJECTED (429 Too Many Requests)
```

Implementation:

```
public class TokenBucketRateLimiter { private final int maxTokens; private final double refillRatePerMs; private double currentTokens; private long lastRefillTimestamp; public TokenBucketRateLimiter(int maxTokens, int refillPerSecond) { this.maxTokens = maxTokens; this.refillRatePerMs = refillPerSecond / 1000.0; this.currentTokens = maxTokens; this.lastRefillTimestamp = System.currentTimeMillis(); } public synchronized boolean allowRequest() { refill(); if (currentTokens >= 1) { currentTokens -= 1; return true; } return false; } private void refill() { long now = System.currentTimeMillis(); double tokensToAdd = (now - lastRefillTimestamp) * refillRatePerMs; currentTokens = Math.min(maxTokens, currentTokens + tokensToAdd); lastRefillTimestamp = now; } }
```

Distributed Rate Limiting (For Paytm Scale)

A single server rate limiter won't work when you have 50 servers behind a load balancer. Use **Redis**:

```
// Using Redis + Lua for atomic token bucket String luaScript = "" local key = KEYS[1] local maxTokens = tonumber(ARGV[1]) local refillRate = tonumber(ARGV[2]) local now = tonumber(ARGV[3]) local bucket = redis.call('HMGET', key, 'tokens', 'lastRefill') local tokens = tonumber(bucket[1]) or maxTokens local lastRefill = tonumber(bucket[2]) or now local elapsed = now - lastRefill tokens = math.min(maxTokens, tokens + elapsed * refillRate) if tokens >= 1 then tokens = tokens - 1 redis.call('HSET', key, 'tokens', tokens, 'lastRefill', now) redis.call('EXPIRE', key, 60) return 1 else redis.call('HSET', key, 'tokens', tokens, 'lastRefill', now) return 0 end "";
```

Quick Comparison

Algorithm	Pros	Cons	Use Case
-----------	------	------	----------

Fixed Window	Simple, low memory	Boundary burst	Internal/simple APIs
Sliding Window Log	Exact	High memory	Low-traffic, strict limits
Token Bucket	Allows bursts, smooth	Slightly complex	Production APIs (Paytm, Stripe)
Sliding Window Counter	Good balance	Approximate	General purpose

3. Design Patterns — Strategy Pattern

What is it?

The Strategy Pattern lets you define a **family of algorithms**, put each in a separate class, and make them **interchangeable at runtime**.

Real World Analogy

Think of how you pay for a Paytm order:

- UPI
- Credit Card
- Paytm Wallet
- Net Banking

Each payment method has different processing logic, but from the order's perspective, it just says "pay this amount" and the strategy handles the rest.

Without Strategy Pattern (The Bad Way)

```
public class PaymentProcessor {
    public void pay(String method, double amount) {
        if (method.equals("UPI")) {
            // UPI-specific logic: validate VPA, call NPCI, etc.
            System.out.println("Paying " + amount + " via UPI");
        } else if (method.equals("CREDIT_CARD")) {
            // Card-specific logic: tokenize, call payment gateway, etc.
            System.out.println("Paying " + amount + " via Credit Card");
        } else if (method.equals("WALLET")) {
            // Wallet-specific logic: check balance, debit, etc.
            System.out.println("Paying " + amount + " via Wallet");
        }
        // Adding a new method? Modify this class. Violates Open/Closed Principle.
    }
}
```

Problems:

- Giant if-else chain grows with every new payment method
- Modifying existing class to add new behavior (violates Open/Closed Principle)
- Hard to test individual strategies
- All logic crammed into one class

With Strategy Pattern (The Right Way)

```
// Step 1: Define the strategy interface
public interface PaymentStrategy {
    void pay(double amount);
    boolean supports(String method);
}
// Step 2: Implement each strategy
@Component
public class UpiPayment implements PaymentStrategy {
    // ...
}
```

```

PaymentStrategy { @Override public void pay(double amount) { // Validate VPA, call NPCI API, handle callback
System.out.println("Processing UPI payment of ■" + amount); } @Override public boolean supports(String
method) { return "UPI".equalsIgnoreCase(method); } } @Component public class CreditCardPayment implements
PaymentStrategy { @Override public void pay(double amount) { // Tokenize card, call payment gateway, verify
3DS System.out.println("Processing Credit Card payment of ■" + amount); } @Override public boolean
supports(String method) { return "CREDIT_CARD".equalsIgnoreCase(method); } } @Component public class
WalletPayment implements PaymentStrategy { @Override public void pay(double amount) { // Check wallet
balance, debit System.out.println("Processing Wallet payment of ■" + amount); } @Override public boolean
supports(String method) { return "WALLET".equalsIgnoreCase(method); } } // Step 3: The context class that
uses strategies @Service public class PaymentProcessor { private final List<PaymentStrategy> strategies;
@Autowired public PaymentProcessor(List<PaymentStrategy> strategies) { this.strategies = strategies; } public
void processPayment(String method, double amount) { PaymentStrategy strategy = strategies.stream() .filter(s
-> s.supports(method)) .findFirst() .orElseThrow(() -> new IllegalArgumentException("Unsupported payment: " +
method)); strategy.pay(amount); } }

```

Adding a new payment method? Just create a new class that implements `PaymentStrategy`. No existing code modified. Spring auto-discovers it via `@Component`.

Amdocs Example You Can Mention

"At Amdocs, we use the Strategy Pattern for offer validation. Different product categories (mobile, broadband, TV) have different validation rules. Instead of a giant switch statement, each product type has its own `ValidationStrategy` implementation. When we onboard a new product category, we just add a new strategy class — no changes to existing code."

4. Design Patterns — Observer Pattern

What is it?

The Observer Pattern defines a **one-to-many dependency** between objects. When one object (the Subject) changes state, all its dependents (Observers) are notified automatically.

Real World Analogy

Think of a YouTube channel:

- The channel is the **Subject**
- Subscribers are the **Observers**
- When a new video is uploaded (state change), all subscribers get notified
- You can subscribe/unsubscribe at any time

Implementation

```

// The event (what happened) public class OrderEvent { private final String orderId; private final String
status; private final double amount; public OrderEvent(String orderId, String status, double amount) {
this.orderId = orderId; this.status = status; this.amount = amount; } // getters } // Observer interface
public interface OrderObserver { void onOrderEvent(OrderEvent event); } // Concrete observers @Component
public class InventoryService implements OrderObserver { @Override public void onOrderEvent(OrderEvent event)
{ if ("CONFIRMED".equals(event.getStatus())) { System.out.println("Reducing inventory for order " +
event.getOrderId()); } } } @Component public class NotificationService implements OrderObserver { @Override
public void onOrderEvent(OrderEvent event) { System.out.println("Sending SMS/email for order " +
event.getOrderId() + " status: " + event.getStatus()); } } @Component public class LoyaltyService implements
OrderObserver { @Override public void onOrderEvent(OrderEvent event) { if
("CONFIRMED".equals(event.getStatus())) { int points = (int)(event.getAmount() / 10);

```

```

System.out.println("Adding " + points + " loyalty points"); } } } // Subject (the thing being observed)
@Service public class OrderService { private final List<OrderObserver> observers; @Autowired public
OrderService(List<OrderObserver> observers) { this.observers = observers; } public void placeOrder(String
orderId, double amount) { // Core order logic System.out.println("Order " + orderId + " placed for ■" +
amount); // Notify all observers OrderEvent event = new OrderEvent(orderId, "CONFIRMED", amount);
observers.forEach(obs -> obs.onOrderEvent(event)); } }

```

Spring's Built-in Observer: ApplicationEvent

Spring has first-class support for the Observer pattern:

```

// Define event public class PaymentCompletedEvent extends ApplicationEvent { private final String orderId;
private final double amount; public PaymentCompletedEvent(Object source, String orderId, double amount) {
super(source); this.orderId = orderId; this.amount = amount; } // getters } // Publisher (Subject) @Service
public class PaymentService { @Autowired private ApplicationEventPublisher publisher; public void
completePayment(String orderId, double amount) { // process payment... publisher.publishEvent(new
PaymentCompletedEvent(this, orderId, amount)); } } // Listeners (Observers) – completely decoupled, no
interface needed @Component public class ReceiptGenerator { @EventListener public void
onPayment(PaymentCompletedEvent event) { System.out.println("Generating receipt for " + event.getOrderId());
} } @Component public class CashbackService { @EventListener public void onPayment(PaymentCompletedEvent
event) { System.out.println("Processing cashback for ■" + event.getAmount()); } } // Async observer (doesn't
block the payment flow) @Component public class AnalyticsService { @Async @EventListener public void
onPayment(PaymentCompletedEvent event) { System.out.println("Logging analytics for " + event.getOrderId()); }
}

```

Why Observer is Everywhere at Paytm

When a payment succeeds at Paytm, many things happen:

1. Receipt is generated
2. Cashback is calculated
3. Loyalty points are added
4. SMS/Push notification is sent
5. Analytics are logged
6. Merchant is notified

Without Observer pattern, the `PaymentService` would need to know about and call all 6 services. With Observer, it just publishes an event and doesn't care who's listening.

Quick Revision Table

Topic	Key Point
Double Booking	Pessimistic = FOR UPDATE lock, Optimistic = version column, Distributed = Redis lock
Rate Limiter	Token Bucket for production (allows bursts), Redis for distributed
Strategy Pattern	Family of interchangeable algorithms, eliminates if-else chains

Observer Pattern	One-to-many notification, Spring's @EventListener is the built-in impl
When to use Pessimistic	Low concurrency, must-not-fail (bank transfers)
When to use Optimistic	High read, low write contention (profiles, carts)